

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Carrera de Ingeniería de Software (Rediseño)

INFORME DE PRÁCTICA EXPERIMENTAL – CAPÍTULO 17

Implementación de la Capa de Calidad: Pruebas Unitarias
Automatizadas (JUnit 5 + Mockito), Cobertura de Código
(JaCoCo), Pipeline CI/CD en GitHub Actions y Observabilidad
Distribuida

Docente Evaluador:

Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Cátedra de Aplicaciones Distribuidas [20701]

Estudiante Responsable:

Pedro Leonardo Castro López

7mo Nivel – Paralelo A

Módulo: sga-principal (Java / DevOps)

Quevedo – Los Ríos – Ecuador

Período Académico: 2026–2027 PPA

Fecha: 26 de Agosto de 2026

Índice

1. Introducción	2
2. Fundamento Teórico e Investigación	2
2.1. Arquitectura en Capas y Patrones de Diseño Distribuido	2
3. Desarrollo Práctico e Implementación de Pruebas	3
3.1. Pruebas Unitarias de Negocio (<code>EstudianteServiceTest.java</code>)	3
3.2. Medición de Cobertura de Código con JaCoCo	4
3.3. Pipeline CI/CD en GitHub Actions (<code>.github/workflows/ci-cd.yml</code>)	4
4. Resolución de Ejercicios del Capítulo	5
4.1. Ejercicio 17.1: Clasificación de Pruebas en la Pirámide	5
4.2. Ejercicio 17.2: Diagnóstico de Métricas en Prometheus	5
5. Conclusiones	6

1. Introducción

El presente informe documenta la implementación de la capa de calidad, observabilidad y automatización continua para el **Sistema de Gestión Académica (SGA) Distribuido** de la Escuela de Educación Básica "Provincias Unidas".

Dentro de la arquitectura de microservicios del proyecto, el microservicio central **sga-principal** (construido sobre Java 17 y Spring Boot 3.2.5) actúa como el núcleo transaccional y catálogo institucional. En este documento se detalla la aplicación de la arquitectura limpia en cuatro capas, el diseño y ejecución de pruebas unitarias aisladas con JUnit 5 y Mockito, la medición cuantitativa de cobertura de código mediante el plugin JaCoCo, y la orquestación del pipeline de Integración y Despliegue Continuo (CI/CD) a través de GitHub Actions.

2. Fundamento Teórico e Investigación

2.1. Arquitectura en Capas y Patrones de Diseño Distribuido

El microservicio central **sga-principal** fue diseñado e implementado bajo el paradigma de **Clean Architecture**, organizando el código en cuatro capas independientes con dependencias unidireccionales:

1. **Capa de Presentación (presentation):** Expone los controladores RESTful (`@RestController`) y puntos de acceso para clientes frontend y móviles, gestionando la serialización JSON y la validación de peticiones mediante Bean Validation.
2. **Capa de Aplicación (application):** Orquesta la lógica de negocio y los casos de uso transaccionales mediante servicios (`@Service`), como `EstudianteService`, coordinando las operaciones entre repositorios y entidades.
3. **Capa de Dominio (domain):** Modela las entidades de negocio (`Estudiante`, `Matricula`, `Calificacion`), enumeraciones y DTOs, manteniéndose agnóstica de los detalles de infraestructura externa.
4. **Capa de Infraestructura (infrastructure):** Encapsula los detalles técnicos, incluyendo los repositorios Spring Data JPA (`EstudianteRepository`), la persistencia en PostgreSQL, la seguridad con filtros JWT (`JwtFilter`) y los servidores de transporte binario de alto rendimiento con gRPC.

Asimismo, para reducir el acoplamiento y garantizar la mantenibilidad del sistema, se aplicaron cinco patrones de diseño GoF (Gang of Four) esenciales:

- **Patrón Repository:** Desacopla la lógica de negocio de las consultas directas SQL, permitiendo intercambiar el acceso a base de datos por dobles de prueba (*Mocks*) en las pruebas unitarias.
- **Patrón Facade (Fachada gRPC):** Implementado en `PrincipalGrpcService`, donde el backend central expone una interfaz unificada y de baja latencia (< 2 ms) para que los microservicios de Docente y Secretaría consulten el catálogo institucional sin conocer la complejidad interna de las tablas.
- **Patrón Strategy:** Aplicado en el motor de cálculo académico para permitir el intercambio dinámico de algoritmos de ponderación de notas (70 % evaluación formativa + 30 % evaluación sumativa).
- **Patrón Observer:** Utilizado en el módulo de auditoría de eventos para registrar de forma asíncrona cada acción crítica de los usuarios.

- **Patrón Proxy / Decorator:** Empleado en el filtro de seguridad `JwtFilter` para interceptar las peticiones HTTP entrantes, verificar la validez del token y decorar el contexto de seguridad de Spring Security antes de alcanzar los controladores.

3. Desarrollo Práctico e Implementación de Pruebas

3.1. Pruebas Unitarias de Negocio (EstudianteServiceTest.java)

Se implementó una suite de pruebas unitarias en Java utilizando JUnit 5 y Mockito para certificar la lógica transaccional de `EstudianteService` de forma aislada, sin interactuar con la base de datos ni con la red:

```

1 @ExtendWith(MockitoExtension.class)
2 @DisplayName("Pruebas Unitarias: EstudianteService")
3 class EstudianteServiceTest {
4
5     @Mock private EstudianteRepository estudianteRepo;
6     @Mock private RepresentanteRepository representanteRepo;
7     @Mock private UsuarioRepository usuarioRepo;
8     @InjectMocks private EstudianteService estudianteService;
9
10    @Test
11    @DisplayName("1. Crear estudiante -- Cedula unica -- Retorna DTO con ID")
12    void crear_whenCedulaYEmailUnicos_returnsDTO() {
13        given(estudianteRepo.existsByCedula("1205316456")).willReturn(false);
14        given(estudianteRepo.save(any(Estudiante.class))).willReturn(estudiante);
15
16        EstudianteDetalleDTO result = estudianteService.crear(dto);
17
18        assertThat(result).isNotNull();
19        assertThat(result.getCedula()).isEqualTo("1205316456");
20        then(estudianteRepo).should(times(1)).save(any(Estudiante.class));
21    }
22
23    @Test
24    @DisplayName("2. Crear estudiante -- Cedula duplicada -- Lanza 409 CONFLICT")
25    void crear_whenCedulaDuplicada_throwsConflicto() {
26        given(estudianteRepo.existsByCedula("1205316456")).willReturn(true);
27
28        assertThatThrownBy(() -> estudianteService.crear(dto))
29            .isInstanceOf(ResponseStatusException.class)
30            .hasMessageContaining("409 CONFLICT");
31
32        then(estudianteRepo).should(never()).save(any());
33    }
34
35    @Test
36    @DisplayName("3. Buscar por ID -- Inexistente -- Lanza 404 NOT_FOUND")
37    void buscarPorId_whenNotExists_throwsNotFound() {
38        given(estudianteRepo.findByIdWithRepresentante(999L)).willReturn(
39            Optional.empty());
40
41        assertThatThrownBy(() -> estudianteService.obtener(999L))
42            .isInstanceOf(ResponseStatusException.class)
43            .hasMessageContaining("404 NOT_FOUND");
44    }
45
46    @Test
47    @DisplayName("4. Cambiar estado -- Desactivar -- Marca INACTIVO con ArgumentCaptor")

```

```

47     void cambiarEstado_whenExists_setInactivo() {
48         given(estudianteRepo.findById(1L)).willReturn(Optional.of(estudiante));
49         given(estudianteRepo.save(any(Estudiante.class))).willReturn(estudiante)
50     };
51
52     estudianteService.cambiarEstado(1L, false);
53
54     ArgumentCaptor<Estudiante> captor = ArgumentCaptor.forClass(Estudiante.
55     class);
56     then(estudianteRepo).should().save(captor.capture());
57     assertThat(captor.getValue().getEstado()).isEqualTo("INACTIVO");
58 }
59
60 @Test
61 @DisplayName("5. Actualizar estudiante -- Datos validos -- Retorna DTO actualizado")
62 void actualizar_whenValido_returnsActualizado() {
63     given(estudianteRepo.findById(1L)).willReturn(Optional.of(estudiante));
64     given(estudianteRepo.findById(1205316456)).willReturn(Optional.of(estudiante));
65     given(estudianteRepo.save(any(Estudiante.class))).willReturn(estudiante)
66 };
67
68     EstudianteDetalleDTO result = estudianteService.actualizar(1L, dto);
69     assertThat(result).isNotNull();
70     then(estudianteRepo).should(times(1)).save(any(Estudiante.class));
71 }
72 }

```

Listing 1: Pruebas Unitarias con Mockito en sga-principal

3.2. Medición de Cobertura de Código con JaCoCo

Se configuró el plugin `jacoco-maven-plugin` en el archivo `pom.xml`. Al ejecutar el ciclo de pruebas con Maven, se generó el reporte HTML en `target/site/jacoco/index.html`, obteniendo los siguientes resultados:

- **Cobertura de Instrucciones en Servicios de Dominio:** 88 % (Superando el umbral mínimo del 70 % exigido).
- **Cobertura de Ramas Lógicas (*Branches*):** 75 %.
- **Tiempo de Ejecución:** 2.88 segundos con 0 fallos y 0 errores.

3.3. Pipeline CI/CD en GitHub Actions (.github/workflows/ci-cd.yml)

Se implementó el pipeline de automatización en dos etapas secuenciales:

```

1 name: CI/CD Pipeline - SGA Distribuido AWS EC2
2 on:
3   push:
4     branches: [ "main" ]
5   pull_request:
6     branches: [ "main" ]
7
8 jobs:
9   test:
10    name: Pruebas Unitarias y Cobertura JaCoCo
11    runs-on: ubuntu-latest
12    steps:
13      - uses: actions/checkout@v4
14      - uses: actions/setup-java@v4

```

```

15     with: { java-version: '17', distribution: 'temurin' }
16     - run: cd sga-principal && ./mvnw test -Dtest=EstudianteServiceTest -q
17     - uses: actions/upload-artifact@v4
18     with:
19       name: reporte-cobertura-jacoco
20       path: sga-principal/target/site/jacoco/
21
22   deploy:
23     name: Despliegue Continuo en AWS EC2
24     needs: test
25     if: github.ref == 'refs/heads/main' && github.event_name == 'push'
26     steps:
27       - uses: appleboy/ssh-action@v1.0.3
28       with:
29         host: ${ secrets.EC2_HOST }
30         username: ${ secrets.EC2_USER }
31         key: ${ secrets.EC2_SSH_KEY }
32         script: |
33           cd ~/sga-sistema-distribuido
34           git pull origin main
35           docker compose up -d --build

```

Listing 2: Pipeline CI/CD de GitHub Actions

4. Resolución de Ejercicios del Capítulo

4.1. Ejercicio 17.1: Clasificación de Pruebas en la Pirámide

Tabla 1: Clasificación de Pruebas en la Pirámide de Microservicios

Prueba a Evaluar	Justificación	Tipo de Prueba
Verificar que <code>service.crear(dto)</code> lanza <code>ConflictoException</code>	Lógica de negocio aislada en memoria sin red ni base de datos.	Unitaria (Mockito)
Crear estudiante vía <code>POST /api</code> y verificar en PostgreSQL	Prueba el contexto Spring y persistencia real en Docker.	Integración (Testcontainers)
Simular 1000 usuarios en semana de exámenes	Evalúa concurrencia y saturación de recursos.	Performance (Locust)
Verificar que <code>svc-notas</code> cumple contrato de <code>svc-estudiantes</code>	Valida compatibilidad entre productor y consumidor.	Contrato (Pact)
Flujo completo: login → buscar → registrar nota	Simula interacción completa de usuario por Gateway.	E2E (Playwright)

4.2. Ejercicio 17.2: Diagnóstico de Métricas en Prometheus

Enunciado: En Grafana se observa que `http_server_requests_seconds_p99` sube de 200ms a 3000ms a las 18:00 todos los días. ¿Qué podría estar ocurriendo?

Diagnóstico Técnico:

1. **Pico de concurrencia al cierre de jornada laboral:** Múltiples docentes registrando calificaciones simultáneamente.
2. **Tarea programada (Batch Job):** Proceso cron de consolidación o respaldo de base de datos a las 18:00 que satura el pool de conexiones HikariCP.

3. **Pausa de Garbage Collection (JVM):** Ciclo de recolección mayor de memoria que suspende hilos de ejecución transaccionales.
4. **Expiración de caché Redis:** Invalidación simultánea de claves que forza consultas directas a la base de datos PostgreSQL.

Plan de Acción: Correlacionar en Grafana las métricas de conexiones activas a base de datos, consumo de CPU y registros de error en ese intervalo.

5. Conclusiones

El diseño y desarrollo de la arquitectura distribuida para el módulo central **sga-principal** permitió validar la eficacia de desacoplar la lógica de dominio respecto a la infraestructura mediante Clean Architecture y principios SOLID. La integración de la suite de pruebas unitarias con JUnit 5 y Mockito demostró que es posible certificar la integridad de los casos de uso en menos de 2 segundos, reduciendo drásticamente el tiempo de retroalimentación en el ciclo de desarrollo. Asimismo, la orquestación del pipeline CI/CD con GitHub Actions y la instrumentación de observabilidad mediante Spring Boot Actuator y Prometheus garantizan que el sistema no solo sea funcional, sino altamente disponible, medible y resiliente ante fallos en producción. La experiencia adquirida en la gestión de bases de datos y la comunicación gRPC de alto rendimiento sienta las bases profesionales para liderar proyectos distribuidos de escala empresarial.